

SMART CONTRACT **AUDIT REPORT**

KDV Staking Smart Contract

DECEMBER 2025

Contents

1. EXECUTIVE SUMMARY	4
1.1 Methodology	4
2. FINDINGS OVERVIEW	7
2.1 Project Info And Contract Address	7
2.2 Summary	7
2.3 Key Findings	8
3. DETAILED DESCRIPTION OF FINDINGS	9
3.1 Commission Limit Bypass During Initialization	9
3.2 setOperator is missing a check for a zero address	12
3.3 ifAdmin Modifier Prevents Users from Querying Pause State	13
4. CONCLUSION	15
5. APPENDIX	16
5.1 Basic Coding Assessment	16
5.1.1 Apply Verification Control	16
5.1.2 Authorization Access Control	16
5.1.3 Forged Transfer Vulnerability	16
5.1.4 Transaction Rollback Attack	17
5.1.5 Transaction Block Stuffing Attack	17
5.1.6 Soft Fail Attack Assessment	17
5.1.7 Hard Fail Attack Assessment	18
5.1.8 Abnormal Memo Assessment	18
5.1.9 Abnormal Resource Consumption	18
5.1.10 Random Number Security	19
5.2 Advanced Code Scrutiny	19
5.2.1 Cryptography Security	19
5.2.2 Account Permission Control	19
5.2.3 Malicious Code Behavior	20
5.2.4 Sensitive Information Disclosure	20
5.2.5 System API	20
6. DISCLAIMER	21
7. REFERENCES	22

8. About Exvul Security

23

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **KDV Staking** to review smart contract implementation. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood:** represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact:** measures the technical loss and business damage of a successful attack.
- **Severity:** determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

		Informational	Low	Medium	High
Likelihood	High	INFO	MEDIUM	HIGH	CRITICAL
	Medium	INFO	LOW	MEDIUM	HIGH
	Low	INFO	LOW	LOW	MEDIUM
		IMPACT			

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs:** We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing:** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations:** We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

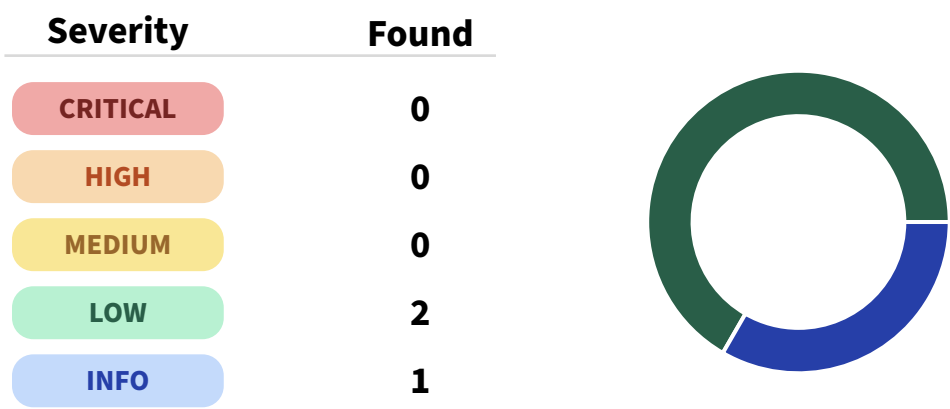
To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
KDV Staking	24/12/2025 - 06/01/2026	Solidity
Repository		
Commit Hash		
20375b91459eb2a26780e4710f4b118d		

2.2 Summary



2.3 Key Findings

Severity	Findings Title	Status
LOW	Commission Limit Bypass During Initialization	Acknowledge
LOW	setOperator is missing a check for a zero address	Acknowledge
INFO	ifAdmin Modifier Prevents Users from Querying Pause State	Acknowledge

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 Commission Limit Bypass During Initialization

SEVERITY:

LOW

STATUS:

Acknowledge

PATH:

KdvStaking.sol

DESCRIPTION:

initialize_2 sets the commission limit without validating that the already configured commission fee from initialize_1 complies with this new limit.

```
// KdvStaking.sol
function setCommissionFee(uint256 _commissionFee) external onlyAdmin {
    if (_commissionFee > KdvStakingStorageLib.getCommissionLimit()) {
        revert FeeOutOfRange();
    }
    KdvStakingStorageLib.setCommissionFee(_commissionFee);
    emit CommissionFeeUpdated(_commissionFee);
}

function initialize_1(
    address _admin,
    address _operator,
    address _treasury,
    address _depositContract,
    address _elDispatcher,
    address _clDispatcher,
    address _feeRecipientImplementation,
    uint256 _commissionFee,
    uint256 commissionLimitBPS
) external init(1) {
    _checkAddress(_admin);
    KdvStakingStorageLib.setAdmin(_admin);
    _checkAddress(_operator);
    KdvStakingStorageLib.setOperator(_operator);
    _checkAddress(_treasury);
    KdvStakingStorageLib.setTreasury(_treasury);

    if (_commissionFee > BASIS_POINTS) {    //@audit Only checks if
fee > 100%, does not check against limit
        revert FeeOutOfRange();
    }
}
```

```
    }
    KdvStakingStorageLib.setCommissionFee(_commissionFee);

    _checkAddress(_elDispatcher);
    KdvStakingStorageLib.setELDispatcher(_elDispatcher);
    _checkAddress(_clDispatcher);
    KdvStakingStorageLib.setCLDispatcher(_clDispatcher);
    _checkAddress(_depositContract);
    KdvStakingStorageLib.setDepositContract(_depositContract);
    _checkAddress(_feeRecipientImplementation);
    KdvStakingStorageLib.setFeeRecipientImplementation(
        _feeRecipientImplementation);
    initialize_2(commissionLimitBPS);
}

function initialize_2(uint256 commissionLimitBPS) public init(2) {
    if (commissionLimitBPS > BASIS_POINTS) {    //@audit Missing
validation current fee < new limit
        revert FeeOutOfRange();
    }
    KdvStakingStorageLib.setCommissionLimit(commissionLimitBPS);
}
}
```

IMPACT:

1. Scenario deployer calls initialize_1 with _commissionFee = 5000 (50%) and commissionLimitBPS = 1000 (10%)
2. State after initialization: commissionFee = 50%, commissionLimit = 10%
3. Protocol operates with 50% fee indefinitely. Admin can only decrease fee, but the limit has already been bypassed

RECOMMENDATIONS:

Add validation to ensure the existing commission fee does not exceed the newly set limit.

```
function initialize_2(uint256 commissionLimitBPS) public init(2) {
    if (commissionLimitBPS > BASIS_POINTS) {
        revert FeeOutOfRange();
    }
+
+   uint256 currentFee = KdvStakingStorageLib.getCommissionFee();
+   if (currentFee > commissionLimitBPS) {
+       revert FeeOutOfRange();
+   }
+
+ }
```

```
}    KdvStakingStorageLib.setCommissionLimit(commissionLimitBPS);
```

3.2 setOperator is missing a check for a zero address

SEVERITY:

LOW

STATUS:

Acknowledge

PATH:

contracts/KdvStaking.sol

DESCRIPTION:

The setOperator method may incorrectly allow the Operator to be set to address(0), potentially posing an operational risk.

```
// contracts/KdvStaking.sol
function setOperator(address _operator) external onlyAdmin {
    KdvStakingStorageLib.setOperator(_operator);
    emit OperatorUpdated(_operator);
}
```

RECOMMENDATIONS:

Add verification mechanism.

```
+ function setOperator(address _operator) external onlyAdmin {
+     _checkAddress(_operator);
    KdvStakingStorageLib.setOperator(_operator);
    emit OperatorUpdated(_operator);
}
```

3.3 ifAdmin Modifier Prevents Users from Querying Pause State

SEVERITY:

INFO

STATUS:

Acknowledge

PATH:

TUPProxy.sol

DESCRIPTION:

isPaused normally designed as a view function can be called by any address. ifAdmin modifier with the Transparent Proxy's delegation mechanism makes the pause state completely opaque to external observers.

```
// TUPProxy.sol
/// @dev Retrieves Paused state
/// @return Paused state
function isPaused() external ifAdmin returns (bool) {
    return StorageSlot.getBooleanSlot(_PAUSE_SLOT).value;
}

/// @dev Pauses system
function pause() external ifAdmin {
    StorageSlot.getBooleanSlot(_PAUSE_SLOT).value = true;
}

/// @dev Unpauses system
function unpause() external ifAdmin {
    StorageSlot.getBooleanSlot(_PAUSE_SLOT).value = false;
}

/// @dev Overrides the fallback method to check if system is not
/// paused before
/// @dev Address Zero is allowed to perform calls even if system is
/// paused. This allows
/// view functions to be called when the system is paused as rpc
/// providers can easily
/// set the sender address to zero.
function _beforeFallback() internal override {
    if (StorageSlot.getBooleanSlot(_PAUSE_SLOT).value == false ||
msg.sender == address(0)) {
        super._beforeFallback();
    } else {
        revert CallWhenPaused();
    }
}
```

IMPACT:

Non admin calls to `isPaused()` always fail.

RECOMMENDATIONS:

Remove the `ifAdmin` modifier.

```
- function isPaused() external ifAdmin returns (bool) {  
+ function isPaused() external view returns (bool) {  
    return StorageSlot.getBooleanSlot(_PAUSE_SLOT).value;  
}
```

4. CONCLUSION

In this audit, we thoroughly analyzed **KDV Staking** smart contract implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

- [1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). <https://cwe.mitre.org/data/definitions/191.html>.
 - [2] MITRE. CWE-197: Numeric Truncation Error. <https://cwe.mitre.org/data/definitions/197.html>.
 - [3] MITRE. CWE-400: Uncontrolled Resource Consumption. <https://cwe.mitre.org/data/definitions/400.html>.
 - [4] MITRE. CWE-440: Expected Behavior Violation. <https://cwe.mitre.org/data/definitions/440.html>.
 - [5] MITRE. CWE-684: Protection Mechanism Failure. <https://cwe.mitre.org/data/definitions/693.html>.
 - [6] MITRE. CWE CATEGORY: 7PK - Security Features. <https://cwe.mitre.org/data/definitions/254.html>.
 - [7] MITRE. CWE CATEGORY: Behavioral Problems. <https://cwe.mitre.org/data/definitions/438.html>.
 - [8] MITRE. CWE CATEGORY: Numeric Errors. <https://cwe.mitre.org/data/definitions/189.html>.
 - [9] MITRE. CWE CATEGORY: Resource Management Errors. <https://cwe.mitre.org/data/definitions/399.html>.
 - [10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology
-

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

Contact

 **Website**
www.exvul.com

 **Twitter**
[@EXVULSEC](https://twitter.com/EXVULSEC)

 **Email**
contact@exvul.com

 **Github**
github.com/EXVUL-Sec